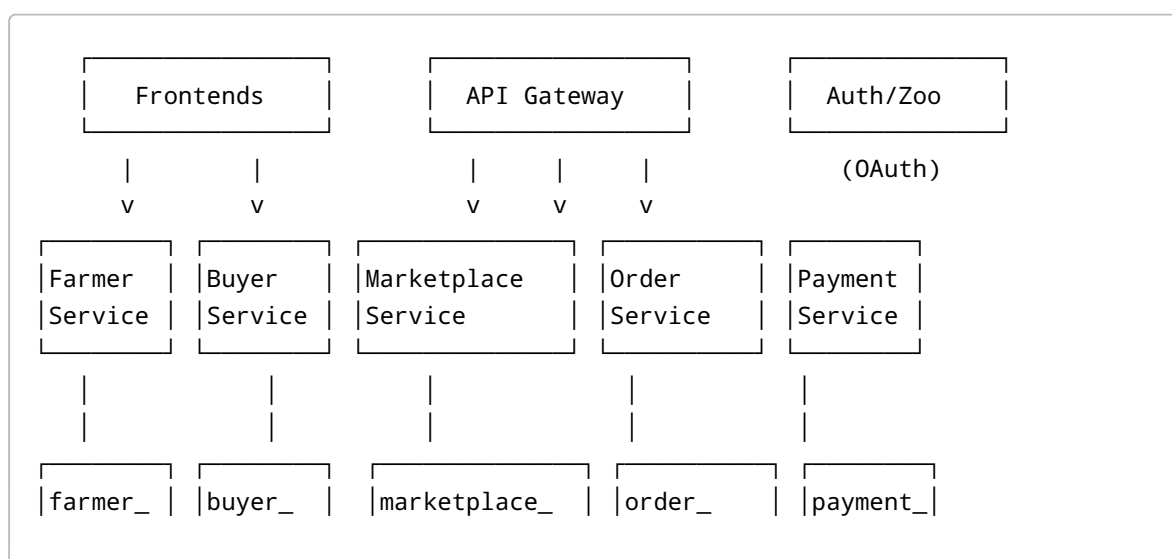


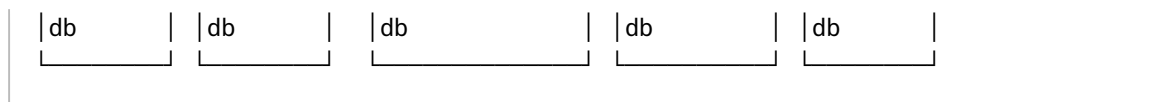
Service Inventory and Boundaries

For a **farmer/tea supply chain platform**, identify all key stakeholders and their data domains. Typically you'd have at least: Farmers (tea smallholders), Buyers (markets, exporters, hotels, supermarkets like Keells/Cargills), Government (agricultural agencies), NGOs (extension services), Transport/Logistics providers, and external data providers (Weather, Quality labs, etc.). Each business capability becomes its own microservice. For example:

- **User Service:** Authentication, user profiles (farmers, buyers, officers).
- **Farmer Service:** Farmer registration, farm details, crop inventory.
- **Buyer/Exporter Service:** Buyer profiles (including corporate buyers like supermarkets, exporters).
- **Marketplace/Product Service:** Crop/product listings, categories, search.
- **Order Service:** Handles orders, quotes, and order tracking.
- **Payment Gateway:** Third-party payment integration (Stripe/PayPal/Ayapay, etc.).
- **Price Service:** Publishes current market prices (could subscribe to government/auction feeds).
- **Government Service:** Manages subsidies, grants, regulations, disease alerts.
- **NGO Service:** Training programs, advisory content, farm assistance.
- **Transport Service:** Manages deliveries, vehicles, drivers, shipment tracking.
- **Notification Service:** Sends emails/SMS/push (e.g. via Twilio or AWS SNS) for alerts (new orders, subsidies, weather warnings).
- **Quality/Certification Service (optional):** Validates crop quality or organic certification.
- **Weather Service:** External API calls for forecasts.

Each of these should be a separately deployable microservice **with its own backend and database**. In a true microservices design, each service “owns” its data [36†L1-L4] [32†L1-L4] . For instance, the Order Service owns the `orders` table in its own `order_db` ; the Farmer Service owns the `farmers` table in `farmer_db` . This ensures loose coupling: “each microservice’s persistent data [is] private to that service and accessible only via its API” [36†L1-L4] . We isolate databases/queues per service (a 12-factor apps practice) [32†L1-L4] .





Each service has its own codebase, runs independently (e.g. in a Docker container), and only exposes well-defined APIs (usually REST/HTTP or gRPC). For example, when a farmer adds a product, the Marketplace Service handles that request and stores it in *marketplace_db*, but it never directly updates *farmer_db*. Any cross-cutting concerns (authentication, rate-limiting, load balancing) are handled at the gateway or platform level.

【35†embed_image】 *Figure: Example microservices architecture. The API Gateway is the single entry point. Client requests (web/mobile app) go to the gateway, which routes them to independent services (Account, Inventory, ShoppingCart, Payment, Shipping). Each service runs separately and has its own database* 【32†L1-L4】 【16†L99-L107】 .

Databases and Data Ownership

Separate database per service: It's a best practice that “a service's transactions only involve its database” 【36†L1-L4】 . You **should not share a single monolithic database** across microservices, because that creates tight coupling. Instead, use one of: private tables per service, or better, a separate schema or database instance per service 【1†L75-L83】 . This makes ownership clear (for example, only the Farmer Service can write to the `farmers` table) and allows using the most suitable database technology for each service (SQL, NoSQL, etc.) 【1†L104-L112】 . Atlassian similarly notes: “Each microservice is composed of a service and database... [we] isolate their resources such as databases... from each other” 【32†L1-L4】 .

Benefits: Loose coupling (one service's schema changes won't break others) 【1†L104-L112】 , independent scaling (each DB can be tuned/sharded as needed) 【1†L54-L60】 , and fault isolation (one DB failure doesn't crash all services).

Trade-offs: You lose easy joins or multi-table transactions across services. Queries that need data from multiple services must be done via API composition or by maintaining separate read-models (CQRS/materialized views) 【1†L129-L139】 . Cross-service *transactions* are implemented via patterns like **Sagas** (with compensation) instead of distributed 2PC 【1†L115-L123】 【6†L101-L110】 . For example, placing an order might involve the Order Service (creating an order) and Payment Service (charging a card). We'd orchestrate that with a Saga (either code or BPMN) rather than a single DB transaction.

Inter-Service Communication

Microservices communicate via **APIs and messaging**. A common setup is an **API Gateway** or BFF (Backend-for-Frontend) that all clients (web, mobile) call. The gateway then routes or aggregates calls to backend services 【16†L99-L107】 . The gateway can also handle auth, rate-limiting, SSL termination, etc. For example, when a user fetches product details, the gateway may call the Marketplace, Price, and Government services (to fetch subsidies) and combine results.

All services expose REST (or gRPC) endpoints. One service calls another via HTTP (synchronous) when immediate feedback is needed. Alternatively, use an **event-driven** approach for asynchronous workflows. For instance, Government Service could emit a `SubsidyPublished` event (to a message

broker like RabbitMQ or Kafka). The Notification Service and Farmer Service could consume this event to send alerts or update data, without blocking the Government Service.

In practice, typical patterns include:

- **REST/gRPC calls:** e.g. Order Service calls Transport Service with `POST /delivery` to book a shipment.
- **Message queues/topics:** e.g. Payment Service publishes `PaymentSucceeded`, Order Service subscribes to finalize the order.
- **API composition:** e.g. a dashboard collects info by calling multiple microservices via the API Gateway, instead of doing a DB join `【1+L129-L139】`.

For reliability, implement retries and circuit-breakers (e.g. Resilience4j, Istio sidecars) around network calls. Use correlation IDs/logging for tracing requests across services.

Service Functions

Each service has a clear responsibility:

- **User/Identity Service:** (Optional) Manages login/auth. It could issue JWT tokens for other services. Otherwise, this logic can live in each service or an API Gateway (e.g. OAuth, Keycloak).
- **Farmer Service:** CRUD on farmer profiles, farm locations, assigned crops. Tracks how much each farmer has.
- **Buyer/Exporter Service:** CRUD on buyer accounts (could include domestic buyers, exporters). Handles business-specific buyer workflows.
- **Marketplace Service:** Allows farmers to list harvests, browse products. Manages inventory listings and search by category.
- **Price Service:** Maintains commodity prices (maybe updated daily). Possibly ingests data from government or auction houses. Provides price lookup for products.
- **Order Service:** Creates and manages orders. When a buyer purchases, it creates an order record, checks inventory (via Marketplace), and calls Payment. Uses Saga to confirm order placement.
- **Payment Gateway:** Could be a third-party API integration. Handles payments/loans. Not your own DB: typically call external API, but you might store transaction records.
- **Transport Service:** Handles delivery logistics. Tracks drivers, vehicles, delivery status. When an order is placed, a message is sent to Transport Service to schedule pickup and delivery.
- **Government Service:** Admin UI for officials to create subsidies, programs, announcements. Exposes APIs (e.g. `GET /subsidies`) so farmers see the latest programs.
- **NGO Service:** Admin UI for NGOs to publish training/workshop info. Farmers fetch e.g. `GET /workshops`.
- **Notification Service:** Listens for events (new order, subsidy, weather alert) and sends SMS/email/push notifications. For example, on `SubsidyCreated` it might SMS all subscribed farmers.
- **Weather Service:** Usually just calls an external API (OpenWeather). Your code can be part of Farmer Service or separate. It provides forecasts to farmers.
- **SMS/Email Service:** Could be part of Notification Service or a separate wrapper around SMS gateways.

Each service's API should be carefully designed. Use OpenAPI/Swagger to define endpoints and JSON models. Keep schemas tight and versioned.

Containers and Orchestration

Package each microservice in a **Docker container** for consistency and portability [38†L818-L826] . Docker ensures each service's environment is isolated: *"Each container runs independently with its own set of dependencies... any changes or issues in one container do not affect others."* [38†L818-L826] . In development, you can use Docker Compose to run all services locally. In production, deploy containers to a **Kubernetes** (or similar) cluster. Kubernetes will handle service discovery, load balancing, auto-scaling and self-healing of pods [38†L818-L826] [38†L912-L920] .

Typical setup:

- **Docker:** Build images for each service (e.g. `docker build -t farmer-service:v1 .`). Use a private container registry to store them.
- **Kubernetes:** Create Deployment and Service manifests. Use an **Ingress** or API Gateway (e.g. Istio, Ambassador, Kong) for external traffic. K8s auto-restarts failed containers and scales them with demand [38†L912-L920] .
- **Helm:** Optionally use Helm charts for easier deployment management across environments (dev/test/prod).

CI/CD Pipeline (Jenkins, etc.)

Set up a CI/CD pipeline so that each time you commit code, services are built, tested, and deployed automatically. A common tool is Jenkins, but you could also use GitLab CI, GitHub Actions, etc. For example, in Jenkins you might have a **multibranch pipeline** per service:

1. **Build:** Checkout code, compile, run unit tests (`mvn clean install` or similar).
2. **Containerize:** Build Docker image and push to registry.
3. **Deploy to Test:** Apply Kubernetes manifests (e.g. `kubectl apply -f k8s/`) to a dev cluster.
4. **Integration Tests:** Run automated integration tests against the deployed service.

One can orchestrate this with Jenkinsfile (pipeline-as-code). For instance: *"The Build stage will invoke `mvn clean install` ... The Deploy stage will invoke `kubectl apply -f deployment.yml` to deploy the service to a Kubernetes cluster."* [13†L269-L275] . Configure Jenkins to trigger on Git commits or pull requests [13†L277-L279] .

Integrating tests: Add JMeter or other performance tests into Jenkins. Jenkins can "trigger jobs by commits" to run JMeter scripts automatically [28†L120-L127] . For example, after deployment Jenkins could run a JMeter job simulating 100 buyers and 50 farmers performing operations. The [CloudBees guide](#) notes *"Jenkins easily enables users to integrate load testing into its workflow..."* [28†L120-L127] and that *"JMeter is one of the most popular open-source load testing systems"* which can be invoked post-build [28†L124-L132] .

Use Jenkins plugins like the Performance Plugin to collect JMeter reports and fail the build if SLA thresholds are not met [28†L129-L134] [28†L170-L178] . Also lint and unit-test at every commit. A typical pipeline might look like:

```
stage('Build') {
    sh 'mvn clean package'
```

```

}
stage('Test') {
    sh 'mvn test'
}
stage('Dockerize') {
    sh 'docker build -t registry/farmer-service:${GIT_SHA} .'
    sh 'docker push registry/farmer-service:${GIT_SHA}'
}
stage('Deploy') {
    sh 'kubectl set image deployment/farmer-service farmer=registry/farmer-
service:${GIT_SHA}'
}
stage('Performance Test') {
    sh 'jmeter -n -t tests/farmer-service.jmx -l results.jtl'
}

```

Include rollback strategies (e.g. use Kubernetes Rollbacks or Helm) in case a deployment fails [【13†L309-L312】](#) .

Monitoring and Logging

With dozens of services, centralized monitoring is essential. Incorporate health checks and metrics (e.g. **Spring Boot Actuator** endpoints) and expose them to a monitoring system. The standard stack is **Prometheus + Grafana + ELK/EFK** (or OpenSearch):

- **Metrics:** Each service exposes metrics (CPU usage, request count, latency) via Prometheus exporters. Prometheus scrapes them and Grafana displays dashboards (e.g. requests/sec per service, error rates). This lets you see if the Order Service is handling 100 req/s or if the Payment Service is erroring. For example, Atlassian notes Docker + Kubernetes “*allows applications to scale up or down based on fluctuating traffic or increased workloads*” [【38†L912-L920】](#) – you would monitor these metrics to trigger scaling.
- **Logging:** Centralize logs with the ELK Stack (Elasticsearch, Logstash/Fluentd, Kibana) or the open-source fork OpenSearch. Each microservice writes JSON logs (include correlation IDs). A log shipper (Fluent Bit) sends logs to Elasticsearch. As Logz.io explains, ELK “*collects and processes data from multiple sources, stores it in one centralized data store*” for analysis [【24†L214-L223】](#) . This way you can search all error logs across services in one place.
- **Alerting:** Configure alerts (e.g. via Prometheus Alertmanager or Grafana) on key metrics (high latency, service down, high error rate).
- **Distributed Tracing:** Consider OpenTelemetry/Jaeger to trace requests through services (so you can see if a request spent too long in the Buyer→Order→Transport chain).

Example Workflow & Failure Scenarios

Consider a typical flow: a farmer lists 100kg of tea leaves in the marketplace. A hotel (buyer) places an order. The sequence is:

1. **Listing:** Farmer’s app calls `POST /harvests` on Marketplace Service. The service saves the listing in `marketplace_db`. (This is an independent transaction in Marketplace Service.)

2. **Ordering:** Buyer's app calls `POST /orders` on Order Service. Order Service creates an order and calls `GET /price/tea` on Price Service to confirm current price. It then calls the Payment Service API to charge the buyer. If payment succeeds, Order Service publishes an `OrderPlaced` event.
3. **Transport:** The Transport Service, subscribed to `OrderPlaced`, schedules a driver and truck. It updates its `transport_db` with a delivery record and status. The farmer and buyer can check delivery status via Transport Service's API.
4. **Notification:** Upon each major event (new order, shipment out for delivery), the Notification Service sends SMS or push notifications to involved parties. For example, Government Service might have just sent a "SubsidyAvailable" event, and Notification Service SMS's all farmers about it.
5. **Monitoring:** Meanwhile, all services emit metrics (via Prometheus) and logs (to ELK). If the Order Service sees a surge, Kubernetes can scale it up automatically (as Docker/K8s allow on-demand scaling [\[38†L912-L920\]](#)).

Failure case: Suppose the Payment Service fails (down or times out). The saga orchestrator (e.g. Camunda/BPMN) triggers compensating actions: the order is canceled and any reserved inventory is freed. Camunda's Zeebe engine can handle this long-running workflow: *"If step three fails, the engine runs compensating actions for steps two and one"* [\[6†L101-L110\]](#) . This ensures partial transactions don't break the business process. Without a saga or orchestration, failures could cause inconsistent state.

Another issue: if one service's database goes down, it only affects that service. Other services continue working. For example, if `price_db` is unreachable, only Price Service and its consumers are affected. The API Gateway could show a meaningful error to the client (fallback to stale price or a default), rather than the entire app crashing. This isolation is why we prefer **separate backends and DBs** [\[36†L1-L4\]](#) [\[32†L1-L4\]](#) .

Tools & Recommendations

- **Camunda** (Zeebe): Use this BPMN/workflow engine to orchestrate complex business processes and sagas [\[6†L101-L110\]](#) . For example, model the order lifecycle (reserve stock, charge payment, dispatch delivery) in a diagram. Camunda will reliably step through each service call and handle retries/compensation if something fails, giving visibility into in-flight processes.
- **Docker:** Containerize every microservice for environment consistency [\[38†L818-L826\]](#) .
- **Kubernetes:** Deploy containers with K8s for orchestration. Use *readiness/liveness probes* so pods auto-restart if stuck, and Horizontal Pod Autoscalers to scale based on CPU/requests [\[38†L912-L920\]](#) .
- **Jenkins:** Implement multibranch pipelines for CI/CD. Integrate JMeter performance tests into the pipeline: "Jenkins easily enables users to integrate load testing into its workflow" [\[28†L120-L127\]](#) , so after each build you can run stress tests.
- **JMeter:** Simulate realistic load (hundreds of farmers & buyers) to ensure the system handles traffic before going live [\[28†L124-L132\]](#) .
- **Monitoring:** As above, use Prometheus/Grafana for metrics and ELK (or hosted SaaS like Logz.io) for logs [\[24†L214-L223\]](#) . Implement alerts (e.g. SMS or Slack via webhook) on critical issues.
- **Notification Tools:** For SMS, consider Twilio or AWS SNS. For in-app notifications, use Firebase Push or AWS SNS push. For emails, use SendGrid or SMTP. These can be wrapped by your Notification Service.
- **Service Mesh** (optional): Tools like Istio or Linkerd can add mutual TLS, advanced traffic routing (canary releases), and built-in telemetry without code changes.

Summary

In summary, the system should not be one monolithic app, but a set of **loosely coupled microservices**. Each has its own repo and database, communicates via APIs/events, and is containerized. An API Gateway fronts them. Use Docker and Kubernetes to run them in a cluster, and Jenkins for automated build/deploy pipelines. Instrument everything for observability and use JMeter to generate realistic traffic.

This approach is more complex than a simple CRUD website, but it matches real-world architectures: “independent services... each service is composed of a service and database” [32†L1-L4] , fronted by a gateway [16†L99-L107] , and managed via CI/CD pipelines [13†L269-L275] [28†L120-L127] . It allows you to simulate a full agricultural ecosystem (farmers, government, buyers, exporters) where each actor has its own service. That will impress your lecturer as a true microservices design rather than a single-site application.

Sources: Authoritative microservices patterns and best practices [36†L1-L4] [16†L99-L107] [38†L818-L826] [13†L269-L275] [28†L120-L127] [6†L101-L110] [24†L214-L223] , and industry guidance (Atlassian, CloudBees, Camunda).
